Auteur: Tran Kenny

Sommaire avec ancres

- Architecture du projet
- Objectif
- Comment lancer le programme
- Les étapes de base de la conception
    - Générer la savane
    - Création des resources
    - Ajout des éléments à la savane
    - Simulation
    - Affichage de la savane
    - Vérifier l'état des animaux
    - Faire manger les animaux
    - Faire deplacer les animaux
    - Générer les ressources
- Bonus
    - Commande de Jeux
    - Sauvegarde de la simulation
    - Reprendre la simulation

Architecture du projet

|
|-- helper.py
|-- main.py
|-- sauvegarde.json

helper.py

Ce fichier stocke tout les fonctions que nous aurons besoin lors du projet.

main.py

Est le fichier permettant permettant de lancer l'application

sauvegarde.json

Est le fichier qui permet de sauvegarder l'etat de la partie.

Objectif

L'objectif du programme est de crée une savane qui va évoluer au fil des jours.
La savane est composé de 3 éléments principaux:

- Les animaux
- Les points d'eau
- Les bosquets

- Les prédateurs

Le comportement attendu par la simulation est le suivant:

- Les animaux sont des entités qui se déplacent dans la savane et qui ont besoin de se nourrir et de boire pour survivre.
- Les points d'eau et les bosquets sont des ressources qui peuvent être consommées par les animaux.
- Les prédateurs sont des animaux qui peuvent attaquer les autres animaux.
- Les animaux peuvent se reproduire et mourir.

Comment lancer le programme

```
$ python main.py
```

Les étapes de base de la conception

Comme indiqué dans le TP on va suivre le comportement attendu par la simulation

Générer la savane

```
def generer_savane() -> tuple[list[list[str]], list[list[int]], int]:
    """
    Retourne la grille de la savane, le tableau d'etat et la taille de la savane
    Returnes:
        grille_savane (list[list[str]])
        tableau_etat (list[list[int]])
        savane (int)
    """
    taille_savane = int(input("choisit la taille de ta savane (entre 10 - 50 et
pair )"))
    while not 10 <= taille_savane <= 50 and taille_savane % 2 != 0:
        taille_savane = int(input("choisit la taille de ta savane (entre 10 - 50
et pair )"))

    grille_savane = [[' ' for _ in range(taille_savane)] for _ in
range(taille_savane)]
    # ecriture sans list comprehension
    # grille_savane = []
    # for _ in range(taille_savane):
    #     ligne = []
    #     for _ in range(taille_savane):
    #         ligne.append(' ')
    #     grille_savane.append(ligne)
    tableau_etat = [[0 for _ in range(taille_savane)] for _ in
range(taille_savane)]
    print(f"la taille de la savane est de :{taille_savane}")

    return grille_savane, tableau_etat, taille_savane
```

exemple de grille_savane (vide) pour une taille de 10

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

exemple de tableau d'état (vide) pour une taille de 10

```
[[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '],
 [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']]
```

Création des resources

Context: Une resource (point d'eau ou bosquet) voit sa quantité entamée à chaque fois qu'un animal consomme cette resource. Une resource dont la quantité atteint 0 est détruite et disparaît définitivement.

On comprend qu'il nous faut un moyen de créer les resources

```python
def generer_ressource(type: str) -> int:
    """
    retourne le nbr de ressource(eau, nourriture)
    Returns:
       ressource (int)
   """
    ressource = int(input(f"choisit le nrb de ressource ({type}): "))
    while not 1 <= ressource <= 10:
        ressource = int(input(f"choisit le nrb de ressource ({type}) : "))
    print(f"vous avez ajouté {ressource}")
    return ressource
```

La fonction ci dessus est la fonction de generation des resources que l'on a fait la dernière fois.

Cette fonction prend en paramètre un **type** eau ou un bosquet, il n'est utilisé que pour faire de l'affichage et savoir qu'est ce que l'on entre comme paramètre. Et retourne le nombre de resource qui à été saisi par l'utilisateur.

```
while not 1 <= ressource <= 10:
```

La boucle while contraint l'utilisateur a saisir un valeur de resource comprise entre 1 et 10 sinon elle continu de demander une valeur.

Une fois la fonction de generation de resource créé on crée celle des animaux.

```
def generer_animaux() -> int:
    """
    retourne le nbr d'animaux
    Returns:
        animaux (int)
    """
    animaux = int(input("choisit le nrb d'animaux : "))
    while not 2 <= animaux <= 20:
        animaux = int(input("choisit le nrb d'animaux : "))
    print(f"vous avez ajouté {animaux}")
    return animaux
```

C'est la meme fonction que `generer_ressource` sauf que la condition de la boucle est entre 2 et 20.

Une fois généré on peut ajouter au programme principal.

```
from helper import generer_animaux, generer_ressource, generer_savane

nbr_animaux = generer_animaux()
nbr_eau = generer_ressource('eau')
nbr_nourriture = generer_ressource('nourriture')
```

Ajout des éléments à la savane

Afin d'améliorer la lisibilité on va créer une fonction principal `ajout_elements` qui va prendre en parametre `grille_savane`, `grille_etat`, `taille_savane`, `nbr_animaux`, `nbr_eau` et `nbr_nourriture`. Cette fonction va permettre de remplir le la grille de la savane de manière aleatoire avec le nombre de resource et animau.

```
def ajout_elements(grille_savane: list[list[str]],
                   grille_etat: list[list[int]],
                   taille_savane: int,
```

```
                    nbr_animaux: int,
                    nbr_eau: int,
                    nbr_nourriture: int):
    _ajout_animaux(grille_savane, taille_savane, nbr_animaux, grille_etat)
    _ajout_eau(grille_savane, taille_savane, nbr_eau, grille_etat)
    _ajout_nourriture(grille_savane, taille_savane, nbr_nourriture, grille_etat)
```

On ajoute _ devant les fonction privé, c'est a dire des fontion que l'on ne va pas utiliser á l'exterieur du fichier **helper.py**.

Les 3 fonctions `_ajout_animaux`, `_ajout_eau`, `_ajout_nourriture` vont permettre de remplir nos tableau `grille_savane` et `grille_etat` défini plus haut.

On va créer un fonction privé `_generer_position` qui va permettre de donner des positions aléatoire. Nous utiliserons la fonction `randrang(x)` qui prend en parametre un entier x. exemple `randrange(10)` retourne un entier compris entre 0 et 9.

```
def _generer_position(taille_savane: int) -> tuple[int, int]:
    return randrange(taille_savane), randrange(taille_savane)
```

Cette fonction retourne 2 chiffres aléatoires représentant x et y. Ils permettront de placer les elements dans un tableau multidimensionnel.

Exemple:

```
x = 0 // ligne 0
y = 2 // colonne 2
valeur_a_ajouter = 1

tableau_multidimensionnel = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0],
]

tableau_multidimensionnel[x][y] = valeur_a_ajouter

# resultat de cette opération
#[
#   [0, 0, 1],
#   [0, 0, 0],
#   [0, 0, 0],
# ]
```

Les fonctions d'ajout

```
def _ajout_animaux(grille_savane, taille_savane, grille_etat, nbr_animaux):
    for i in range(nbr_animaux):
        ligne, colonne = _generer_position(taille_savane)
        while grille_savane[ligne][colonne] != " ":
            ligne, colonne = _generer_position(taille_savane)
        grille_savane[ligne][colonne] = "A"
        grille_etat[ligne][colonne] = 3


def _ajout_eau(grille_savane, taille_savane, grille_etat, nbr_eau):
    for i in range(nbr_eau):
        ligne, colonne = _generer_position(taille_savane)
        while grille_savane[ligne][colonne] != " ":
            ligne, colonne = _generer_position(taille_savane)
        grille_savane[ligne][colonne] = "E"
        grille_etat[ligne][colonne] = 2


def _ajout_nourriture(grille_savane, taille_savane, grille_etat, nbr_nourriture):
    for i in range(nbr_nourriture):
        ligne, colonne = _generer_position(taille_savane)
        while grille_savane[ligne][colonne] != " ":
            ligne, colonne = _generer_position(taille_savane)
        grille_savane[ligne][colonne] = "N"
        grille_etat[ligne][colonne] = 1
```

Chaque fonction possède la signature suivante:

```
def _ajout_{element}(grille_savane, taille_savane, grille_etat, nbr_{element}):
```

Elle ce ressemble et nous allons essayer d'éclaircir ce qu'elle font.

Prenons la 1er ligne, elle défini une boucle qui va ajouter le nombre d'element désiré dans la `grille_savane` et leur etat dans `grille_etat`.

```
for i in range(nbr_{elements}):
```

La ligne 2, permet de générer les positions pour placer les elements 1 par 1 avec la fonction `_generer_position`

```
ligne, colonne = _generer_position(taille_{element})
```

Ligne 3-4, si la case n'est pas rempli on doit chercher une autre position pour placer nos éléments. **Context:** Deux élèments de l'environement ne peuvent occuper la meme case simultanément.

```
while grille_savane[ligne][colonne] != " ":
        ligne, colonne = _generer_position(taille_{element})
```

ligne 5-6, Elles pemettre d'ajouter la valeur de chaque case.

```
grille_savane[ligne][colonne] = {valeur_de_la_case} # "A" (animaux), "E" (eau),
"N" (nourriture)
grille_etat[ligne][colonne] = {les_pdv_de_la_resource}
```

Simulation

La simulation est une boucle comprise entre 10 et 1000 jours et si la totalité des animaux meurt avant la fin de la simulation, la simulation s'arrête.

```
nb_jour_max_simulation = _definir_nombre_de_jour_simulation()
jour_de_depart = 0

for jour in range(jour_de_depart, nb_jour_max_simulation):
```

La fonction `_definir_nombre_de_jour_simulation` va permettre à l'utilisateur de définir le nombre de jour de la simulation.

```
def _definir_nombre_de_jour_simulation() -> int:
    """
    Retourne le nombre de jour de simulation
    Returns:
        animaux (int)
    """
    jour = int(input("choisit le nrb de jour de simulation :"))
    while not 10 <= jour <= 1000:
        jour = int(input("choisit le nrb de jour de simulation :"))
    print(f"vous avez ajouté {jour} jour de simulation")
    return jour
```

Cette fonction est similaire à `generer_animaux` et `generer_ressource` sauf que la condition de la boucle est entre 10 et 1000.

Affichage de la savane

```
_afficher_information(grille_savane, jour)
```

La fonction `_afficher_information` va permettre d'afficher les informations de la savane.
Elle prend en parametre `grille_savane` et `jour` qui est le nombre de jour de la simulation.
Et va afficher la grille de la savane et le nombre de jour de la simulation.

```
def _afficher_information(grille_savane, jour):
    """
    Affiche les informations de la savane
    Args:
        grille_savane (list[list[str]])
        jour (int)
    Returns:
        None
    """
    print(f"jour {jour}")
    pprint(grille_savane)
    nbr_animaux = 0
    nbr_eau = 0
    nbr_nourriture = 0
    for ligne in grille_savane:
        for value in ligne:
            if value == "A":
                nbr_animaux += 1
            if value == "E":
                nbr_eau += 1
            if value == "N":
                nbr_nourriture += 1

    print(f"il y a : {nbr_animaux} animaux")
    print(f"il y a :{nbr_eau} point d'eau ")
    print(f"il y a :{nbr_nourriture} nourriture")
```

La fonction `_afficher_information` va afficher le nombre d'animaux, de point d'eau et de nourriture.

Vérifier l'état des animaux

```
_verifier_etat_animaux(grille_savane, grille_etat)
```

Cette fonction va permettre de vérifier l'état des animaux, afin de savoir si les animaux sont morts (de soif ou de faim).

```
def _verifier_etat_animaux(grille_savane, grille_etat, jour):
    """
    Si un animal n'a pas mangé pendant 10 jours il meurt
    Si un animal n'a pas bu pendant 3 jours il meurt
    """
    for ligne, value_ligne in enumerate(grille_savane):
        for colonne, value_colonne in enumerate(value_ligne):
            if value_colonne == "A":
```

```
                type_animal = TYPE_ANIMAUX[grille_etat[ligne][colonne]['type']]
                if (
                    grille_etat[ligne][colonne]["jour_sans_manger"] == 10
                ):
                    grille_savane[ligne][colonne] = " "
                    grille_etat[ligne][colonne] = 0
                    print(
                        f"l'animal en position {ligne}, {colonne} ({type_animal})
est mort de faim le jour {jour}"
                    )
                if (
                    grille_etat[ligne][colonne] != 0 and grille_etat[ligne]
[colonne]["jour_sans_boire"] == 3
                ):
                    grille_savane[ligne][colonne] = " "
                    grille_etat[ligne][colonne] = 0
                    print(
                        f"l'animal en position {ligne}, {colonne} ({type_animal})
est mort de soif le jour {jour}"
                    )
```

La fonction `_verifier_etat_animaux` va permettre de vérifier l'état des animaux, si ils n'ont pas mangé pendant 10 jours ils meurent de faim et si ils n'ont pas bu pendant 3 jours ils meurent de soif. Lors de la mort d'un animal on remplace la case de l'animal par un espace vide et on remet l'état de la case à 0.

Exemple:
On a pris le cas ou l'animal n'a pas mangé pendant 10 jours.

Avant la mort de l'animal
Grille d'état

```
[
    [{
        "jour_sans_boire": 0,
        "jour_sans_manger": 10,
        "type": 0
    }, 0, 0],
    [0, 0, 0],
    [0, 0, 0],
]
```

Grille savane

```
[
    ["A", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "],
]
```

Après la mort de l'animal Grille d'état

```
[
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0],
]
```

Grille savane

```
[
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "],
]
```

Faire manger les animaux

Dans la simulation nous pouvons faire manger les animaux, pour cela nous allons créer une fonction `_faire_manger_animaux` qui va permettre de faire manger les animaux.

```
def _faire_manger_animaux(grille_savane, grille_etat):
    """
    Fonction qui permet de faire manger les animaux
    """
    for ligne, value_ligne in enumerate(grille_savane):
        for colonne, value_colonne in enumerate(value_ligne):
            if (
                value_colonne == "A"
            ):
                # On regarde autour de l'animal
                eau, nourriture = _recherche_autour_animaux(
                    grille_savane, ligne, colonne
                )
                # ...
```

Explication de enumerate, elle permet de parcourir une liste et de retourner l'index et la valeur de chaque élément de la liste.

Exemple:

```
liste = ["a", "b", "c"]
for index, value in enumerate(liste):
    print(index, value)
# elle affiche
# 0 a
```

```
# 1 b
# 2 c
```

A l'intérieur de `_faire_manger_animaux` on parcours la grille de la savane et on regarde si l'animal a trouvé de la nourriture ou de l'eau.

```
def _recherche_autour_animaux(grille_savane, ligne, colonne):
    """
    Retourne la position de l'eau et de la nourriture autour de l'animal
    Args:
        grille_savane (list[list[str]])
        ligne (int)
        colonne (int)
    Returns:
        eau (tuple[int, int])
        nourriture (tuple[int, int])
    """
    position_resource = {"nourriture": [], "eau": []}
    # On parcours les lignes autour de l'animal
    for i in range(-1, 2):
        # On parcours les colonnes autour de l'animal
        for j in range(-1, 2):
            # On verifie que la position n'est pas en dehors de la grille
            print("_recherche_autour_animaux", ligne+i , "i", i, "ligne", ligne)
            if 0 <= ligne + i < len(grille_savane) and 0 <= colonne + j < len(
                grille_savane
            ):
                # On verifie si la position est de la nourriture
                if grille_savane[ligne + i][colonne + j] == "N":
                    position_resource["nourriture"].append((ligne + i, colonne +
j))
                # On verifie si la position est de l'eau
                if grille_savane[ligne + i][colonne + j] == "E":
                    position_resource["eau"].append((ligne + i, colonne + j))

    # retourne le premier element de la nourriture et de l'eau si les taille du
tableau == 0 on return None
    eau = position_resource["eau"][0] if len(position_resource["eau"]) > 0 else
None
    nourriture = (
        position_resource["nourriture"][0]
        if len(position_resource["nourriture"]) > 0
        else None
    )
    return eau, nourriture
```

Explication:

```
for i in range(-1, 2):
```

Cette boucle permet de parcourir les lignes autour de l'animal, la range est de -1 à 2, cela permet de parcourir les lignes autour de l'animal. On imbrique 2 boucles pour parcourir les lignes et les colonnes autour de l'animal.

```
 if 0 <= ligne + i < len(grille_savane) and 0 <= colonne + j < len(grille_savane):
```

la condition permet de vérifier que la position n'est pas en dehors de la grille.

```
if grille_savane[ligne + i][colonne + j] == "N":
    position_resource["nourriture"].append((ligne + i, colonne + j))

if grille_savane[ligne + i][colonne + j] == "E":
    position_resource["eau"].append((ligne + i, colonne + j))
```

cette condition permet de vérifier si la position est de la nourriture ou de l'eau et de l'ajouter dans le tableau `position_resource`.

On va retouner le première élément de la nourriture et de l'eau si les tailles du tableau == 0 on return None.

Suite de la fonction `_faire_manger_animaux`

```
if nourriture:
    # On decremnete la nourriture dans la grille d'etat
    grille_etat[nourriture[0]][nourriture[1]] -= 1
    # Si la nourriture est a 0 on la retire de la grille
    if grille_etat[nourriture[0]][nourriture[1]] == 0:
        grille_savane[nourriture[0]][nourriture[1]] = " "
    # On remet le compteur de jour sans manger a 0
    grille_etat[ligne][colonne]["jour_sans_manger"] = 0
else:
    # On incremente le compteur de jour sans manger
    grille_etat[ligne][colonne]["jour_sans_manger"] += 1
# Si l'animal a trouvé de l'eau
if eau:
    # On decremnete l'eau dans la grille d'etat
    grille_etat[eau[0]][eau[1]] -= 1
    # Si l'eau est a 0 on la retire de la grille
    if grille_etat[eau[0]][eau[1]] == 0:
        grille_savane[eau[0]][eau[1]] = " "
    # On remet le compteur de jour sans boire a 0
    grille_etat[ligne][colonne]["jour_sans_boire"] = 0
else:
    # On incremente le compteur de jour sans boire
    grille_etat[ligne][colonne]["jour_sans_boire"] += 1
```

Les deux conditions permettent de vérifier si l'animal a trouvé de la nourriture ou de l'eau.
Si l'animal a trouvé de la nourriture on décrémente la nourriture dans la grille d'état et on remet le compteur de jour sans manger à 0. Si l'animal n'a pas trouvé de la nourriture on incrémente le compteur de jour sans manger.
Et meme chose pour l'eau.

Exemple: Dans le cas ou un animal n'a pas bu pendant 2 jours et qu'il trouve de l'eau.

```
grille_savane = [
    [" ", " ", "E"],
    [" ", "A", " "],
    [" ", " ", " "],
]
```

```
grille_etat = [
    [0, 0, 1],
    [0, {"jour_sans_boire": 2, "jour_sans_manger": 0}, 0],
    [0, 0, 0],
]
```

Après avoir trouvé de l'eau

```
grille_savane = [
    [" ", " ", " "],
    [" ", "A", " "],
    [" ", " ", " "],
]
```

```
grille_etat = [
    [0, 0, 0],
    [0, {"jour_sans_boire": 0, "jour_sans_manger": 0}, 0],
    [0, 0, 0],
]
```

Faire deplacer les animaux

Cette fonction va permettre de déplacer les animaux de 2 cases adjacente aléatoirement.

```
def _on_deplace_animaux(grille_savane, grille_etat):
    """
    Fonction qui permet de deplacer les animaux de 2 cases adjacente aleatoirement
    """
    for ligne, value_ligne in enumerate(grille_savane):
```

```
        for colonne, value_colonne in enumerate(value_ligne):
            if value_colonne == "A":
                # Il peuvent ce deplacer dans un carre de 3x3 atour de leur
position
                # On genere un deplacement aleatoire
                ligne_deplacement = randrange(-2, 3)
                colonne_deplacement = randrange(-2, 3)
                # On verifie que la position n'est pas en dehors de la grille
                # ou n'est pas occupé sinon on recommence
                while (
                    not 0 <= ligne + ligne_deplacement < len(grille_savane)
                    or not 0 <= colonne + colonne_deplacement < len(grille_savane)
                    or grille_savane[ligne + ligne_deplacement][
                        colonne + colonne_deplacement
                    ]
                    != " "
                ):
                    ligne_deplacement = randrange(-2, 3)
                    colonne_deplacement = randrange(-2, 3)

                # On deplace l'animal sur la nouvelle position et dans la grille
d'etat
                grille_savane[ligne + ligne_deplacement][
                    colonne + colonne_deplacement
                ] = "A"
                grille_etat[ligne + ligne_deplacement][
                    colonne + colonne_deplacement
                ] = grille_etat[ligne][colonne]
                grille_savane[ligne][colonne] = " "
                grille_etat[ligne][colonne] = 0
```

Explication:

```
ligne_deplacement = randrange(-2, 3)
colonne_deplacement = randrange(-2, 3)
```

On génère un déplacement aléatoire pour les lignes et les colonnes. la range est de -2 à 3, cela permet de parcourir les lignes et les colonnes autour de l'animal.

Générer les ressources

```
def _on_regenerer_ressource(grille_savane, grille_etat, jour):
    """
    Fonction qui permet de regenerer les ressources
    1 par jour pour l'eau
    1 tous les 3 jours pour la nourriture
    """
    for ligne, value_ligne in enumerate(grille_savane):
        for colonne, value_colonne in enumerate(value_ligne):
```

```
            if value_colonne == "E":
                grille_etat[ligne][colonne] += 1
            if value_colonne == "N" and jour % 3 == 0:
                grille_etat[ligne][colonne] += 1
```

Cette fonction va permettre de regénérer les ressources, 1 par jour pour l'eau et 1 tous les 3 jours pour la nourriture.

Bonus

Commande de Jeux

Nous allons ajouter des commandes de jeux pour permettre à l'utilisateur de controler la simulation.

```
COMMANDE_NEXT = "N"
COMMANDE_STOP = "S"
COMMNADE_SAUVEGARDER = "SA"
COMMANDE_REPRENDRE = "R"
```

Les commandes de jeux sont les suivantes:

- `N` pour passer au jour suivant
- `S` pour arreter la simulation
- `SA` pour sauvegarder la simulation
- `R` pour reprendre la simulation

On permet a l'utilisateur de saisir une commande de jeux lors de la simulation.

```
# On demande à l'utilisateur de continuer ou d'arreter la simulation
status_simulation = input(
    f"- Pour passer au jour suivant taper {COMMANDE_NEXT} \
                        - Pour arreter taper {COMMANDE_STOP} \
                        - Pour sauvegarder et quitter taper
{COMMNADE_SAUVEGARDER}"
)

while status_simulation not in [
    COMMANDE_NEXT,
    COMMANDE_STOP,
    COMMNADE_SAUVEGARDER,
]:
    status_simulation = input(
        f"- Pour passer au jour suivant taper {COMMANDE_NEXT} \
                        - Pour arreter taper {COMMANDE_STOP} \
                        - Pour sauvegarder et quitter taper
{COMMNADE_SAUVEGARDER}"
    )
```

Sauvegarde de la simulation

Elle va permettre de sauvegarder les données de la simulation en json.
Qui va créer un fichier `sauvegarde.json` qui va contenir les données de la simulation.

```
def _sauvegarde(grille_savane, grille_etat, jour_de_depart,
nb_jour_max_simulation):
    """
    Fonction qui permet de sauvegarder les données de la simulation
    """
    data = {
        "grille_savane": grille_savane,
        "grille_etat": grille_etat,
        "jour": jour_de_depart,
        "nb_jour_max_simulation": nb_jour_max_simulation,
    }
    with open("sauvegarde.json", "w") as file:
        json.dump(data, file, indent=4)
```

On a mis le indent=4 pour que le fichier json soit plus lisible.

Reprendre la simulation

La fonction de reprendre la simulation.
Cette portion de code se trouve dans le fichier `main.py`, elle permet de reprendre la simulation si une sauvegarde existe.

```
    if os.path.exists("sauvegarde.json"):
        status_simulation = input(
            f"Une sauvegarde existe voulez vous la reprendre ?
{COMMANDE_REPRENDRE} / {COMMANDE_STOP}"
        )
        while status_simulation not in [COMMANDE_REPRENDRE, COMMANDE_STOP]:
            status_simulation = input(
                f"Une sauvegarde existe voulez vous la reprendre ?
{COMMANDE_REPRENDRE} / {COMMANDE_STOP}"
            )
        if status_simulation == COMMANDE_REPRENDRE:
            with open("sauvegarde.json", "r") as file:
                # parse json to get values
                data = json.load(file)
                grille_savane = data["grille_savane"]
                grille_etat = data["grille_etat"]
                jour_de_depart = data["jour"]
                nb_jour_max_simulation = data["nb_jour_max_simulation"]
```

L'acouplement des animaux

La fonction `_faire_acouplement_animaux` va permettre de faire acoupler les animaux.

```
def _accouple_animaux(grille_savane, grille_etat):
    """
    Fonction qui permet de faire accoupler les animaux
    A chaque tour de boucle si un animal de type herbivore est a coté d'un autre
animal de type herbivore de sexe opposé
    """
    for ligne, value_ligne in enumerate(grille_savane):
        for colonne, value_colonne in enumerate(value_ligne):
            if value_colonne == "A" and grille_etat[ligne][colonne]["type"] ==
TYPE_HERBIVORE:
                # On regarde autour de l'animal
                _, _, position_animaux, _ = _recherche_autour_animaux(
                    grille_savane, ligne, colonne
                )
                # on selectionne un herbivore de sexe opposé qui n'a pas 10 en
faim et 3 en soif
                partenaire = None
                for animal in position_animaux:
                    if (
                        grille_etat[animal[0]][animal[1]]["type"] ==
TYPE_HERBIVORE
                        and grille_etat[animal[0]][animal[1]]["sexe"]
                        != grille_etat[ligne][colonne]["sexe"]
                        and grille_etat[animal[0]][animal[1]]["jour_sans_manger"]
< 10
                        and grille_etat[animal[0]][animal[1]]["jour_sans_boire"] <
3
                    ):
                        partenaire = animal
                        break
                if partenaire is not None:
                    # On ajoute un nouvel animal sur un case libre sinon ne pas
faire de reproduction
                    _, _, _, position_libre = _recherche_autour_animaux(
                        grille_savane, ligne, colonne
                    )
                    if len(position_libre) > 0:
                        grille_savane[position_libre[0][0]][position_libre[0][1]]
= "A"
                        grille_etat[position_libre[0][0]][position_libre[0][1]] =
{
                            "sexe": "M" if randrange(2) == 0 else "F",
                            "type": TYPE_HERBIVORE,
                            "jour_sans_manger": 0,
                            "jour_sans_boire": 0,
                        }
```

Cette fonction m'a fait modifier la fonction `_recherche_autour_animaux` pour qu'elle retourne les positions libres autour de l'animal et les animaux autour de l'animal.
J'ai du ajouter aussi dans la structure de l'animal le sexe et le type de l'animal.
Voici la nouvelle fonction `_recherche_autour_animaux`:

```
def _recherche_autour_animaux(grille_savane, ligne, colonne):
    """
    Retourne la position de l'eau et de la nourriture autour de l'animal
    Args:
        grille_savane (list[list[str]])
        ligne (int)
        colonne (int)
    Returns:
        eau (tuple[int, int])
        nourriture (tuple[int, int])
    """
    position_resource = {"nourriture": [], "eau": [], "animal": [], "libre": []}
    # On parcours les lignes autour de l'animal
    for i in range(-1, 2):
        # On parcours les colonnes autour de l'animal
        for j in range(-1, 2):
            # On verifie que la position n'est pas en dehors de la grille
            print("_recherche_autour_animaux", ligne+i , "i", i, "ligne", ligne)
            if 0 <= ligne + i < len(grille_savane) and 0 <= colonne + j < len(
                grille_savane
            ):
                # On verifie si la position est de la nourriture
                if grille_savane[ligne + i][colonne + j] == "N":
                    position_resource["nourriture"].append((ligne + i, colonne +
j))
                # On verifie si la position est de l'eau
                if grille_savane[ligne + i][colonne + j] == "E":
                    position_resource["eau"].append((ligne + i, colonne + j))
                # On verifie si la position est un autre animal
                if grille_savane[ligne + i][colonne + j] == "A":
                    position_resource["animal"].append((ligne + i, colonne + j))
                # On verifie si la position est libre
                if grille_savane[ligne + i][colonne + j] == " ":
                    position_resource["libre"].append((ligne + i, colonne + j))

    # retourne le premier element de la nourriture et de l'eau si les taille du
tableau == 0 on return None
    eau = position_resource["eau"][0] if len(position_resource["eau"]) > 0 else
None
    nourriture = (
        position_resource["nourriture"][0]
        if len(position_resource["nourriture"]) > 0
        else None
    )
    return eau, nourriture, position_resource["animal"],
position_resource["libre"]
```

la nouvelle structure d'etat de l'animal

```
{
    "sexe": "M" or "F",
    "type": TYPE_HERBIVORE or TYPE_CARNIVORE,
    "jour_sans_manger": 0,
    "jour_sans_boire": 0,
}
```

Les animaux carnivores

Dans la fonction `_faire_manger_animaux` on va ajouter une condition pour les animaux carnivores.

```
 elif (
                value_colonne == "A"
                and grille_etat[ligne][colonne]["type"] == TYPE_CARNIVORE
            ):
                eau, _, position_animaux, _ = _recherche_autour_animaux(
                    grille_savane, ligne, colonne
                )
                if eau:
                    # On decremnete l'eau dans la grille d'etat
                    grille_etat[eau[0]][eau[1]] -= 1
                    # Si l'eau est a 0 on la retire de la grille
                    if grille_etat[eau[0]][eau[1]] == 0:
                        grille_savane[eau[0]][eau[1]] = " "
                    # On remet le compteur de jour sans boire a 0
                    grille_etat[ligne][colonne]["jour_sans_boire"] = 0
                else:
                    # On incremente le compteur de jour sans boire
                    grille_etat[ligne][colonne]["jour_sans_boire"] += 1
                # mange que les herbivores
                if len(position_animaux) > 0:
                    # On prend un animal qui est hebivore sinon rien
                    animal_a_manger = None
                    for animal in position_animaux:
                        if grille_etat[animal[0]][animal[1]]["type"] ==
TYPE_HERBIVORE:
                            animal_a_manger = animal
                            break
                    if animal_a_manger is None:
                        grille_etat[ligne][colonne]["jour_sans_manger"] += 1
                        continue
                    # On le retire de la grille
                    grille_savane[animal_a_manger[0]][animal_a_manger[1]] = " "
                    grille_etat[animal_a_manger[0]][animal_a_manger[1]] = 0
                    # On remet le compteur de jour sans manger a 0
                    grille_etat[ligne][colonne]["jour_sans_manger"] = 0
                    # Afficher quel animal est devorer avec son type et quel
animal la devorer
                    print(
                        f"l'animal en position {animal_a_manger[0]},
{animal_a_manger[1]} a été devorer par l'animal en position {ligne}, {colonne}"
```

```
                    )
                else:
                    # On incremente le compteur de jour sans manger
                    grille_etat[ligne][colonne]["jour_sans_manger"] += 1
```

Dans la fonction `ajout_elements` on va ajouter des animaux carnivores.

```
# on ajoute 1/3 de carnivores proportionnellement au nombre d'herbivores
    nb_carnivores = randrange(int(nbr_animaux / 3))
    _ajout_animaux(
        grille_savane, taille_savane, grille_etat, nb_carnivores, TYPE_CARNIVORE
    )
    print(f"il y a {nb_carnivores} carnivores qui son apparu dans la savane")
```

Elle va ajouter 1/3 de carnivores proportionnellement au nombre d'herbivores.

Conclusion

Le code final de la simulation

```
def simulation(grille_savane, grille_etat, jour_de_depart,
nb_jour_max_simulation):
    # verifier si les données de la sauvegarde existe
    if jour_de_depart is None and nb_jour_max_simulation is None:
        nb_jour_max_simulation = _definir_nombre_de_jour_simulation()
        jour_de_depart = 0

    for jour in range(jour_de_depart, nb_jour_max_simulation):
        # On affiche les informations de la savane
        _afficher_information(grille_savane, jour)
        # On demande à l'utilisateur de continuer ou d'arreter la simulation
        status_simulation = input(
            f"- Pour passer au jour suivant taper {COMMANDE_NEXT} \
                                - Pour arreter taper {COMMANDE_STOP} \
                                - Pour sauvegarder et quitter taper
{COMMNADE_SAUVEGARDER}"
        )

        while status_simulation not in [
            COMMANDE_NEXT,
            COMMANDE_STOP,
            COMMNADE_SAUVEGARDER,
        ]:
            status_simulation = input(
                f"- Pour passer au jour suivant taper {COMMANDE_NEXT} \
                                - Pour arreter taper {COMMANDE_STOP} \
                                - Pour sauvegarder et quitter taper
{COMMNADE_SAUVEGARDER}"
            )
```

```
        # Si l'utilisateur veut sauvegarder et quitter
        if status_simulation == COMMNADE_SAUVEGARDER:
            _sauvegarde(grille_savane, grille_etat, jour, nb_jour_max_simulation)
            return False

        # Si l'utilisateur veut arreter la simulation
        if status_simulation == COMMANDE_STOP:
            return False
        # On verifie l'etat des animaux
        _verifier_etat_animaux(grille_savane, grille_etat, jour)
        # On fait manger les animaux
        _faire_manger_animaux(grille_savane, grille_etat)
        # On deplace les animaux
        _on_deplace_animaux(grille_savane, grille_etat)
        _accouple_animaux(grille_savane, grille_etat)
        # On regenere les ressources
        _on_regenerer_ressource(grille_savane, grille_etat, jour)
```

le code dans le fichier `main.py` est le suivant:

```
import sys
import os
import json
from helper import (
    ajout_elements,
    COMMANDE_NEXT,
    COMMANDE_REPRENDRE,
    COMMANDE_STOP,
    generer_animaux,
    generer_ressource,
    generer_savane,
    simulation,
)


def main():
    jour_de_depart = None
    nb_jour_max_simulation = None
    status_simulation = COMMANDE_NEXT
    # verifier si une sauvegarde existe
    # Si oui demander si l'utilisateur veut reprendre la simulation
    if os.path.exists("sauvegarde.json"):
        status_simulation = input(
            f"Une sauvegarde existe voulez vous la reprendre ?
{COMMANDE_REPRENDRE} / {COMMANDE_STOP}"
        )
        while status_simulation not in [COMMANDE_REPRENDRE, COMMANDE_STOP]:
            status_simulation = input(
                f"Une sauvegarde existe voulez vous la reprendre ?
{COMMANDE_REPRENDRE} / {COMMANDE_STOP}"
            )
        if status_simulation == COMMANDE_REPRENDRE:
```

```
            with open("sauvegarde.json", "r") as file:
                # parse json to get values
                data = json.load(file)
                grille_savane = data["grille_savane"]
                grille_etat = data["grille_etat"]
                jour_de_depart = data["jour"]
                nb_jour_max_simulation = data["nb_jour_max_simulation"]

    if status_simulation in [COMMANDE_NEXT, COMMANDE_STOP]:
        nbr_animaux = generer_animaux()
        nbr_eau = generer_ressource('eau')
        nbr_nourriture = generer_ressource('nourriture')
        grille_savane, grille_etat, taille_savane = generer_savane()
        ajout_elements(
            grille_savane, grille_etat, taille_savane, nbr_animaux, nbr_eau,
nbr_nourriture
        )

    simulation(grille_savane, grille_etat, jour_de_depart, nb_jour_max_simulation)


if __name__ == "__main__":
    sys.exit(main())
```

Explication:

```
if __name__ == "__main__":
    sys.exit(main())
```

Cette condition permet de vérifier si le fichier est exécuté directement ou importé. Le sys.exit permet de quitter le programme avec le code de retour de la fonction main.

Difficultés rencontrées

La difficulter que j'ai rencontré est de faire fonctionner la fonction `_recherche_autour_animaux` pour qu'elle retourne les positions libres autour de l'animal et les animaux autour de l'animal. Il y a aussi l'attribution de valeurs au animaux sous forme de dictionnaire.
Il y a aussi la gestion des commandes de jeux pour la sauvegarde et la reprise de la simulation.
Analyser le document pour comprendre comment fonctionne une simulation car je comptais tout faire d'un coup.

Amélioration

Je ne sais pas si il y avait une meilleur approche pour l'attributions des valeurs aux animaux.

Ce que j'ai appris

Ce projet ma permis de mieux comprendre comment fonctionne une simulation, la recherche d'information dans les list multi-dimensionnel , aussi la gestion des commandes de jeux pour la sauvegarde et la reprise de

la simulation et l'utilisation des dictionnaires.